

Guia Completo: Arquitetura e Gerenciamento de Memória em C/C++

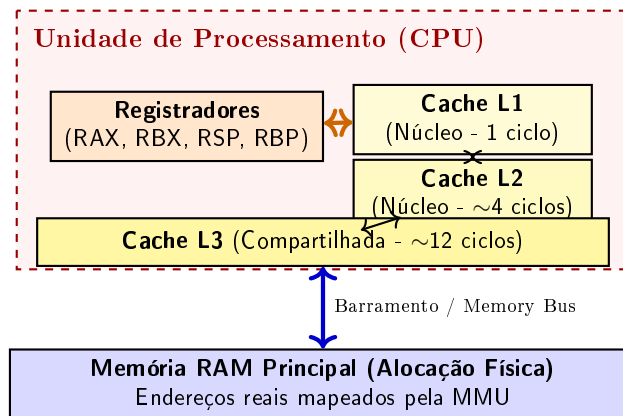
Do Hardware aos Smart Pointers - Aprofundamento Prático e Visual

Conteúdo

1	Visão de Hardware: CPU e Hierarquia de Memória	2
2	Memória Virtual: A Abstração do Sistema Operacional	2
3	O que são Ponteiros?	2
4	Arrays e Aritmética de Ponteiros	4
5	Layout Completo da Memória de um Processo	5
6	Alocação na Pilha (Stack)	6
7	Alocação Dinâmica (Heap) em C	8
8	Estruturas Dinâmicas: Lista Encadeada no Heap	9
9	Gerenciamento Moderno em C++: RAII e Smart Pointers	10

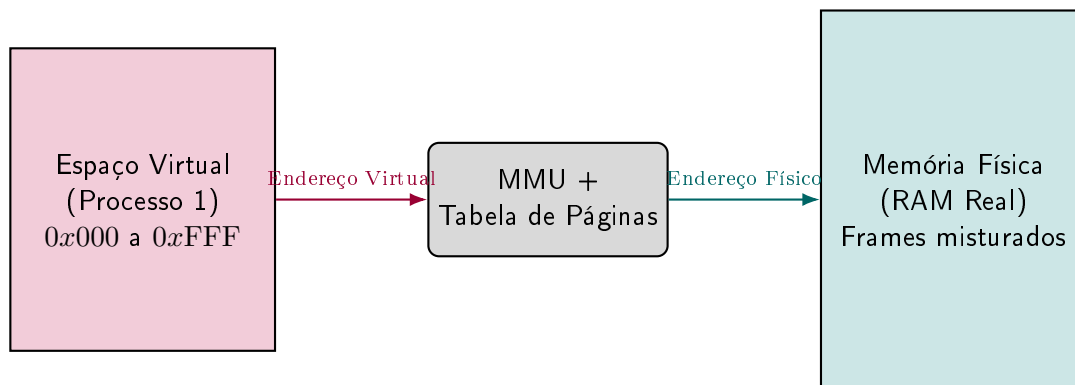
1 Visão de Hardware: CPU e Hierarquia de Memória

Para entender por que o **Stack** é tão rápido e o **Heap** é mais lento, é vital compreender como a CPU interage fisicamente com a memória. A CPU utiliza uma hierarquia de caches extremamente rápidas para mitigar o gargalo de latência do barramento de sistema RAM.



2 Memória Virtual: A Abstração do Sistema Operacional

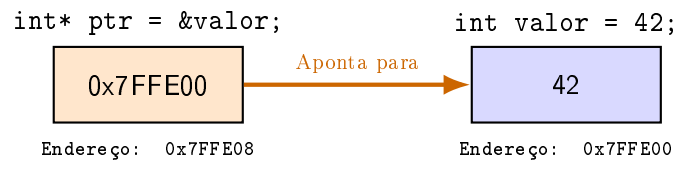
A **Memória Virtual** é uma ilusão criada pelo Sistema Operacional. Quando você programa em C/C++, os ponteiros *não* apontam para o chip físico de RAM, mas sim para **endereços virtuais**. O hardware (*MMU - Memory Management Unit*) e o SO traduzem essas páginas virtuais para *frames* na RAM física. Isso isola processos, evita corrupção cruzada e permite usar o HD/SSD como extensão da RAM (*Swap*).



- **Stack (Pilha):** localidade espacial perfeita - os *Prefetchers* mantêm as linhas sempre nas caches **L1/L2**, daí sua velocidade.
- **Heap (Monte):** alocação esparsa gera *Cache Misses* constantes, forçando buscas na RAM principal.

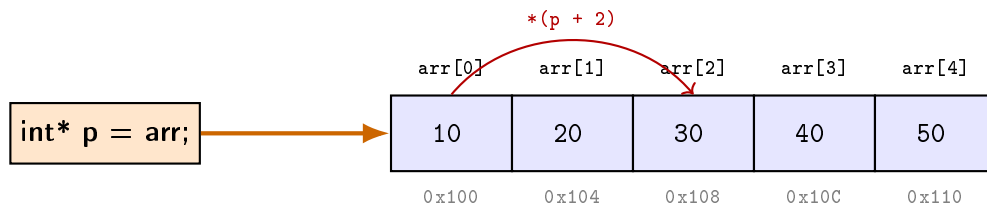
3 O que são Ponteiros?

Um ponteiro é simplesmente uma variável cujo valor é o **endereço de memória** de outra variável. Se a memória é uma rua de casas (bytes), o ponteiro é um papel com o número de uma dessas casas.



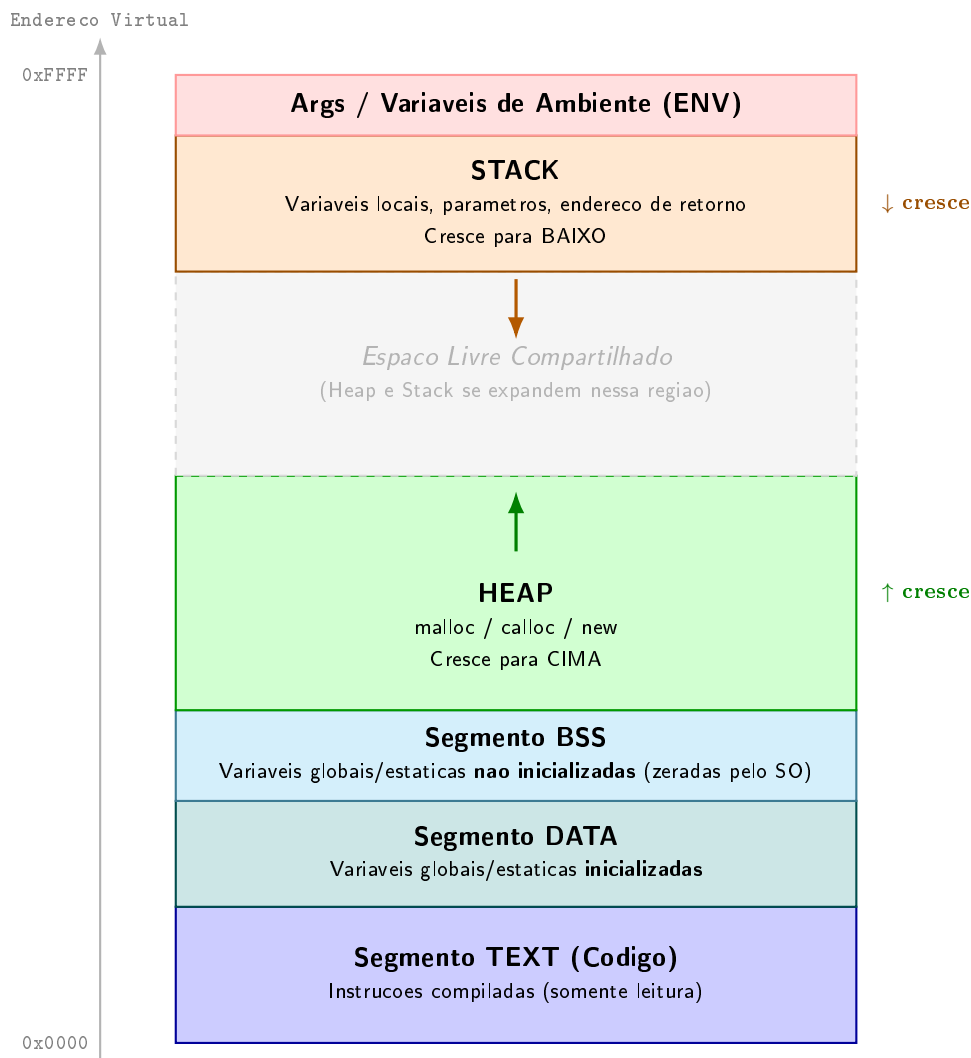
4 Arrays e Aritmética de Ponteiros

Um **array** em C/C++ é um bloco *contíguo* de memória: o identificador decai em ponteiro para o primeiro elemento. A indexação é açúcar sintático para aritmética de ponteiros: `arr[i]  $\equiv$  *(arr + i)`.



5 Layout Completo da Memória de um Processo

Quando o SO carrega um programa, ele divide o espaço virtual em **segmentos fixos e dinâmicos**. O ponto central é que **Heap e Stack compartilham o mesmo espaço livre** — crescendo em sentidos opostos. Se se encontrarem, o programa crasha (*stack overflow* ou *out of memory*).



Exemplo de Código Mapeado para Cada Segmento

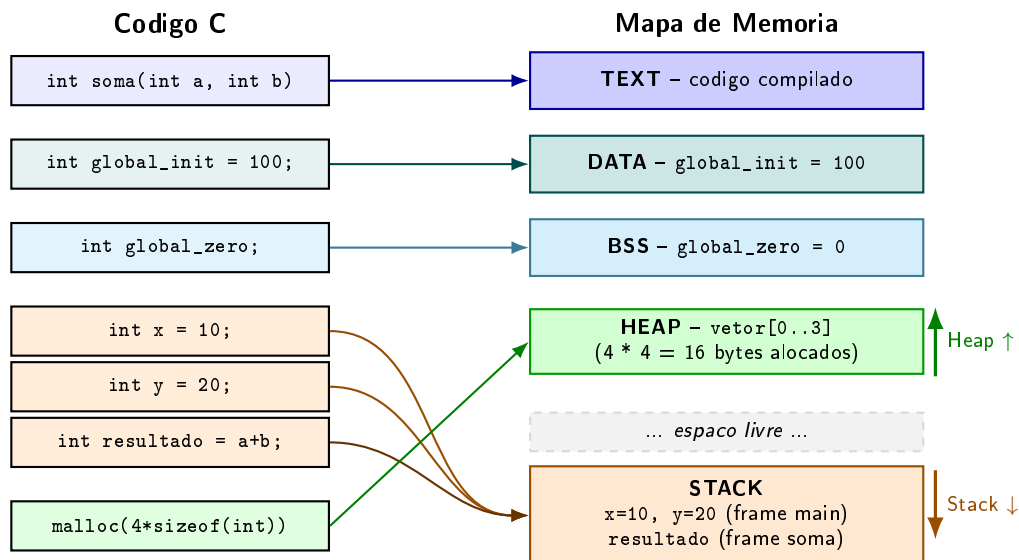
O programa abaixo exercita todos os segmentos. Cada variável está anotada com seu destino na memória:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 /* ---- SEGMENTO DATA: global inicializada ---- */
5 int global_init = 100;
6
7 /* ---- SEGMENTO BSS: global nao inicializada (SO zera) ---- */
8 int global_zero;
9
10 int soma(int a, int b) {           /* TEXT: codigo da funcao */
11     int resultado = a + b;         /* STACK: variavel local do frame soma() */
12     return resultado;
13 }
14
15 int main() {
```

```

16  /* STACK: variaveis locais do frame main() */
17  int x = 10;
18  int y = 20;
19
20  /* HEAP: bloco alocado dinamicamente */
21  int* vetor = (int*) malloc(4 * sizeof(int));
22  vetor[0] = x;
23  vetor[1] = y;
24  vetor[2] = global_init;      /* le do DATA */
25  vetor[3] = soma(x, y);      /* empilha frame soma() na STACK */
26
27  /* imprime enderecos para confirmar os segmentos */
28  printf("TEXT (funcao soma): %p\n", (void*)soma);
29  printf("DATA (global_init): %p\n", (void*)&global_init);
30  printf("BSS (global_zero): %p\n", (void*)&global_zero);
31  printf("STACK (x local): %p\n", (void*)&x);
32  printf("HEAP (vetor[0]): %p\n", (void*)&vetor[0]);
33
34  free(vetor); /* devolve bloco ao Heap */
35  return 0;
36 }

```



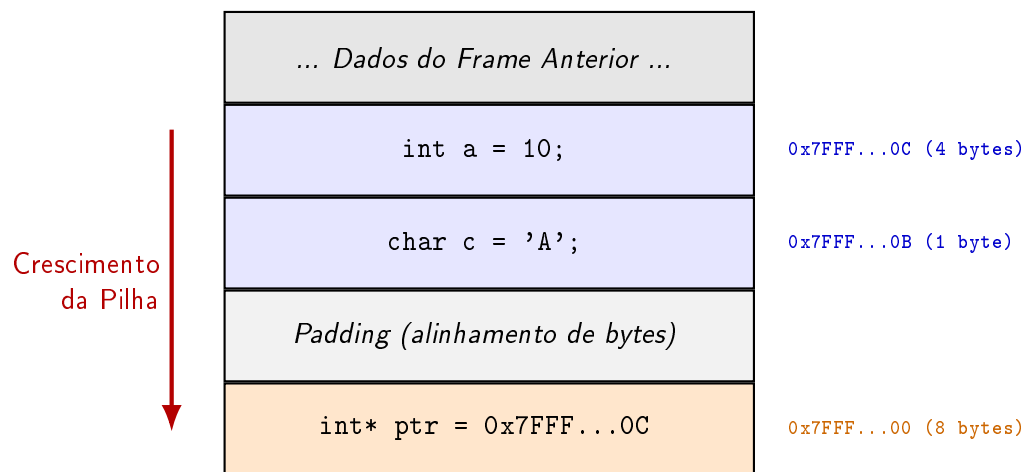
6 Alocação na Pilha (Stack)

Na pilha, as variáveis locais são empilhadas de forma contígua ao chamar a função. O compilador sabe os tipos em tempo de compilação, calcula o tamanho exato do *Stack Frame* e move o *Stack Pointer* (*rsp*) de uma só vez.

```

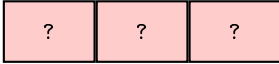
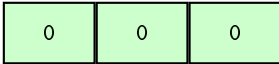
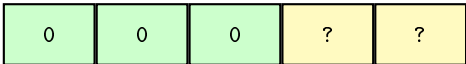
1 void minha_funcao() {
2     int a = 10;    // 4 bytes
3     char c = 'A'; // 1 byte (alinhado para 4 ou 8)
4     int* ptr = &a; // 8 bytes (64 bits) - aponta para 'a'
5 }

```



7 Alocação Dinâmica (Heap) em C

O *Heap* não é estruturado automaticamente. Pedimos espaço ao SO via `<stdlib.h>`. O controle de tempo de vida é **manual**: o que você aloca, você deve liberar com **free** - do contrário ocorre **Memory Leak**.

	<code>int* p = malloc(3*sizeof(int))</code>	
malloc(size)		
Aloca bloco, NÃO inicializa (contém lixo).		
	<code>int* p = calloc(3, sizeof(int))</code>	
calloc(num, size)		
Aloca bloco contíguo e zera os bits.		
	<code>p = realloc(p, 5*sizeof(int))</code>	
realloc(ptr, new_size)		
Redimensiona. Copia dados se mudar de endereço.		
	<code>free(p);</code>	
free(ptr)		
Devolve a memória ao SO. O ponteiro fica "solto" (Dangling).		

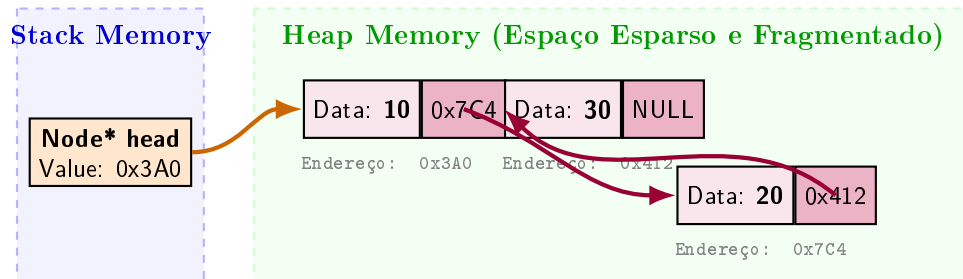
Exemplo Completo em Código:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     // 1. calloc: Array de 3 inteiros, todos zerados.
6     int* array = (int*) calloc(3, sizeof(int));
7
8     // 2. Usando o array (Heap)
9     array[0] = 10; array[1] = 20; array[2] = 30;
10
11    // 3. realloc: Precisamos de espaço para 5 inteiros!
12    // Se não houver espaço logo após o bloco atual,
13    // ele busca novo bloco, copia os 3 antigos e libera o anterior.
14    array = (int*) realloc(array, 5 * sizeof(int));
15    array[3] = 40;
16    array[4] = 50;
17
18    // 4. free: Vital liberar ao fim.
19    free(array);
20    array = NULL; // Boa prática para evitar Dangling Pointers
21
22    return 0;
23 }
```


8 Estruturas Dinâmicas: Lista Encadeada no Heap

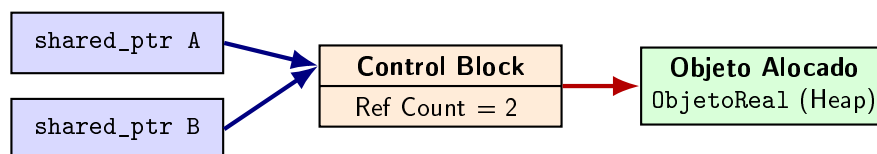
Diferente dos arrays, os nós de uma **Linked List** não são contíguos. São alocados individualmente, espalhados por endereços imprevisíveis (gerando *fragmentação externa*). Cada nó carrega seu dado e um **ponteiro** para o próximo.

O diagrama abaixo mostra como um ponteiro no **Stack** (**head**) serve de ponto de entrada para navegar pelos nós espalhados no **Heap**:



9 Gerenciamento Moderno em C++: RAII e Smart Pointers

Para erradicar *Memory Leaks*, o C++ moderno adota o princípio **RAII (Resource Acquisition Is Initialization)**: os recursos do Heap são encapsulados em objetos cujo escopo é gerenciado automaticamente pelo Stack. Quando o objeto sai de escopo, o destrutor libera o recurso.



Tipos de Smart Pointers:

- `unique_ptr` - posse exclusiva; destruído ao sair do escopo. Zero overhead.
- `shared_ptr` - posse compartilhada via contagem de referências (*ref count*).
- `weak_ptr` - observador não-proprietário; evita ciclos de referência.

```
1 #include <iostream>
2 #include <memory>
3
4 class Recurso {
5 public:
6     Recurso() { std::cout << "Adquirido!\n"; }
7     ~Recurso() { std::cout << "Libertado automaticamente!\n"; }
8 };
9
10 int main() {
11     // unique_ptr: recurso liberado ao sair do bloco - sem vazamentos!
12     {
13         std::unique_ptr<Recurso> ptr = std::make_unique<Recurso>();
14     } // destrutor chamado aqui
15
16     // shared_ptr: dois donos, liberado quando o ultimo sair de escopo
17     {
18         std::shared_ptr<Recurso> a = std::make_shared<Recurso>();
19         std::shared_ptr<Recurso> b = a; // ref count = 2
20     } // ref count cai a 0 -> destrutor chamado
21
22     return 0;
23 }
```